

The Burrows-Wheeler Transform and the FM index

CMSC 423

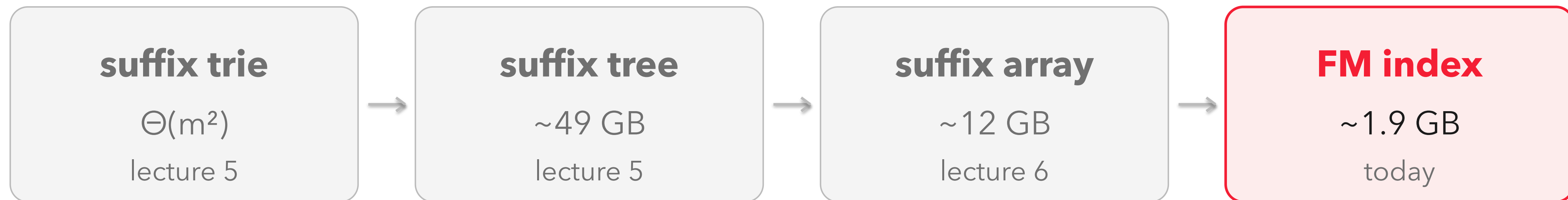
An index that is a permutation of the text itself



Follows the treatments of Ben Langmead (langmead-lab.org/teaching-materials)
and Carl Kingsford. Figures redrawn.

Where we have got to

Four ways to hold every suffix of T. Each one throws away more than the last.



- The suffix array kept the tree's *order* and threw away its pointers. Today we throw away the suffixes themselves.
- The FM index stores a **permutation of T** – one character per text character – plus a handful of sparse side tables, and searches inside it backwards.
- *Forms the basis of many modern short-read aligners*

Notation, and the two strings we will use

T is the text, $m = |T|$, and T always ends in a sentinel $\$$ that is smaller than every other character. P is the pattern we are looking for.

T
a b a a b a \$
 $m = 7$

*the same string as lectures 5 and 6 – small
enough to draw every row of the matrix*

T'
a b a a b a a b a a b b b a a \$
 $m = 16$

*sixteen rows – big enough to use it to
demonstrate sampling*

- Rows of the matrix and entries of the suffix array are numbered from 0. A range of rows is written half-open, $[lo, hi)$, so an empty range is one where $lo = hi$.
- Subscripts on characters are *occurrence numbers*, not part of the text. They are a bookkeeping device we introduce in a few slides and never actually store.

The transform

A reversible permutation of the characters of T

The transform

Reversing it

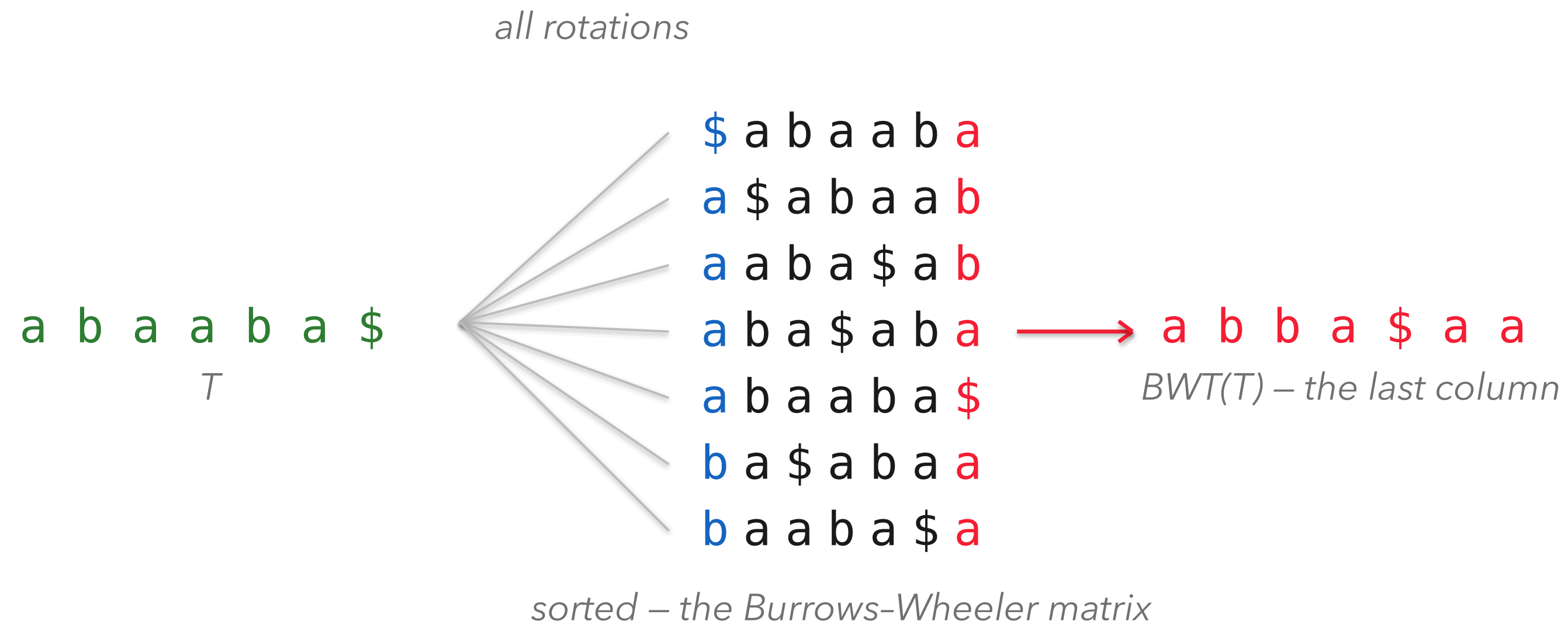
Backward search

Making it fast

In practice

Burrows-Wheeler transform

Write down every rotation of T , sort them, and read off the last column.



Burrows M, Wheeler DJ. A block-sorting lossless data compression algorithm. Digital Equipment Corporation Technical Report 124, 1994.

The Burrows-Wheeler matrix

Sorting the rotations gives the **Burrows-Wheeler matrix** BWM(T). Its first column is written F and its last column L . The transform is just L .

F								L
\$	a	b	a	a	b	a		
a	\$	a	b	a	a	b		
a	a	b	a	\$	a	b		
a	b	a	\$	a	b	a		
a	b	a	a	b	a	\$		
b	a	\$	a	b	a	a		
b	a	a	b	a	\$	a		

$BWM(T)$

BWT(T) = a b b a \$ a a

Same characters as T , in a different order – a permutation, nothing more.

Why would anyone do this?

Because sorting the rotations sorts each character by what follows it – by its **right-context**. Characters that appear in similar contexts end up adjacent in *L*, so *L* has long runs even when *T* does not.

T abaabaabaabaabaabaabaabaabaabaabaabaabaabaabaabaaba\$

42 runs

BWT(T) abbbbbbbbbbbbbbbbbbaaaaaaaaaaaaaaaaaa\$aaaaaaaaaaaaaaaaaaaaa

5 runs

The text has 42 runs of equal characters; its transform has 5. Run-length coding, which does nothing useful to *T*, now has something to bite on. Nothing has been lost – we will get *T* back on the next few slides.

The matrix you have already built

Rotations and suffixes sort into the **same order**. Everything before the **\$** in a rotation is the suffix; everything after it is the wrap-around, and no comparison ever reaches it, because **\$** is unique and smallest.

F *L*

\$	a	b	a	a	b	a
a	\$	a	b	a	a	b
a	a	b	a	\$	a	b
a	b	a	\$	a	b	a
a	b	a	a	b	a	\$
b	a	\$	a	b	a	a
b	a	a	b	a	\$	a

BWM(T)

SA(T)

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

Row *i* of the matrix and entry *i* of the suffix array describe the same suffix.
The suffix array says where it starts; the matrix shows what it looks like.

BWT from the suffix array

Which gives a definition that needs no matrix at all:

$$\text{BWT}[i] = \begin{cases} T[\text{SA}[i] - 1] & \text{if } \text{SA}[i] > 0 \\ \$ & \text{if } \text{SA}[i] = 0 \end{cases}$$

“the character just to the left of each suffix, in sorted order”

SA(T)		L
6	\$	a
5	a \$	b
2	a a b a \$	b
3	a b a \$	a
0	a b a a b a \$	\$
4	b a \$	a
1	b a a b a \$	a

The BWT is a column of characters; the suffix array is a column of integers. On a genome that is two bits per base against thirty-two – a factor of sixteen.

Building it

Given the suffix array, the transform is one pass:

O(m) once you have SA

BWT(T):

SA $\leftarrow$ suffix_array(T)

for i in 0 .. m-1:

 if SA[i] = 0: emit \$

 else: emit T[SA[i] - 1]

- So anything that builds a suffix array builds a BWT. Lecture 6's DC3 does it in $O(m)$ time and $O(m)$ space, though with large constants.
- There are also direct $O(m)$ BWT algorithms that never materialise the suffix array, and blockwise methods that trade time for a much smaller working set – this is what makes indexing a genome on a laptop possible.
- *Kärkkäinen J. Fast BWT in small space by blockwise suffix sorting. Theoretical Computer Science 387(3):249-257, 2007.*

Three questions

Does it compress?

Yes – L groups characters by right-context, which is what run-length and entropy coders want.

answered

Is it reversible?

Yes – and the argument that shows it also turns out to be the search algorithm.

next

Is it an index?

Yes – this is the FM index, and it is the reason we are spending a lecture on it.

after that

The surprise is that the second and third questions have the same answer.

Reversing it

The LF mapping: one idea, two payoffs



Step one: number the occurrences

Give every character in T a rank: how many times that character has already appeared. Call this the **T-ranking**.

a_0	b_0	a_1	a_2	b_1	a_3	\$
0	1	2	3	4	5	6

These subscripts are never stored. They exist so that we can say “the same a ” out loud while looking at two different columns.

Now re-write the matrix carrying the ranks along.

The matrix, ranked

Look only at the first and last columns.

<i>F</i>							<i>L</i>
\$	a_0	b_0	a_1	a_2	b_1	a_3	
a_3	\$	a_0	b_0	a_1	a_2	b_1	
a_1	a_2	b_1	a_3	\$	a_0	b_0	
a_2	b_1	a_3	\$	a_0	b_0	a_1	
a_0	b_0	a_1	a_2	b_1	a_3	\$	
b_1	a_3	\$	a_0	b_0	a_1	a_2	
b_0	a_1	a_2	b_1	a_3	\$	a_0	

F and L are the only two columns we will ever need to look at. Everything between them is here to explain the sort order, and then it goes away – and in a few slides F goes away too.

The *a*s appear in the same order in *F* and in *L*

<i>F</i>						<i>L</i>
\$	<i>a</i> ₀	<i>b</i> ₀	<i>a</i> ₁	<i>a</i> ₂	<i>b</i> ₁	<i>a</i> ₃
<i>a</i> ₃	\$	<i>a</i> ₀	<i>b</i> ₀	<i>a</i> ₁	<i>a</i> ₂	<i>b</i> ₁
<i>a</i> ₁	<i>a</i> ₂	<i>b</i> ₁	<i>a</i> ₃	\$	<i>a</i> ₀	<i>b</i> ₀
<i>a</i> ₂	<i>b</i> ₁	<i>a</i> ₃	\$	<i>a</i> ₀	<i>b</i> ₀	<i>a</i> ₁
<i>a</i> ₀	<i>b</i> ₀	<i>a</i> ₁	<i>a</i> ₂	<i>b</i> ₁	<i>a</i> ₃	\$
<i>b</i> ₁	<i>a</i> ₃	\$	<i>a</i> ₀	<i>b</i> ₀	<i>a</i> ₁	<i>a</i> ₂
<i>b</i> ₀	<i>a</i> ₁	<i>a</i> ₂	<i>b</i> ₁	<i>a</i> ₃	\$	<i>a</i> ₀

as down *F*: *a*₃, *a*₁, *a*₂, *a*₀ as down *L*: *a*₃, *a*₁, *a*₂, *a*₀

Not a coincidence, and not special to this string.

The bs appear in the same order in F and in L

<i>F</i>							<i>L</i>
\$	a_0	b_0	a_1	a_2	b_1	a_3	
a_3	\$	a_0	b_0	a_1	a_2	b_1	
a_1	a_2	b_1	a_3	\$	a_0	b_0	
a_2	b_1	a_3	\$	a_0	b_0	a_1	
a_0	b_0	a_1	a_2	b_1	a_3	\$	
b_1	a_3	\$	a_0	b_0	a_1	a_2	
b_0	a_1	a_2	b_1	a_3	\$	a_0	

bs down F: b_1, b_0 bs down L: b_1, b_0

Not a coincidence, and not special to this string.

Why the orders must agree

Rows beginning with **a** are sorted by what comes after that **a**. Rows ending with **a** are rotations of those same rows by one position – so they are sorted by what comes after that same **a** too. Same keys, same order.

F

```
$ a0 b0 a1 a2 b1 a3
a3 $ a0 b0 a1 a2 b1
a1 a2 b1 a3 $ a0 b0
a2 b1 a3 $ a0 b0 a1
a0 b0 a1 a2 b1 a3 $
b1 a3 $ a0 b0 a1 a2
b0 a1 a2 b1 a3 $ a0
```

sorted by that a's right-context

L

```
$ a0 b0 a1 a2 b1 a3
a3 $ a0 b0 a1 a2 b1
a1 a2 b1 a3 $ a0 b0
a2 b1 a3 $ a0 b0 a1
a0 b0 a1 a2 b1 a3 $
b1 a3 $ a0 b0 a1 a2
b0 a1 a2 b1 a3 $ a0
```

sorted by that a's right-context

However we choose to number occurrences of a character, the numbers come out in the same order down *F* as down *L*. That statement is the whole lecture.

The LF mapping

The i th occurrence of a character c in L and the i th occurrence of c in F are the same occurrence in T .

- It gives us a function on rows. If row i ends in c , then $LF(i)$ is the row that *begins* with that same occurrence of c .
- Stepping from row i to $LF(i)$ moves one character to the LEFT in T . That is why the index walks backwards – and why it is called LF: Last to First.
- Computing it needs two things: where each character's block starts in F , and how many c s precede row i in L . We will make both $O(1)$.

A better numbering

The T-ranking was arbitrary. Rank each character instead by its order of occurrence in L – the **B-ranking** – and F's ranks come out in ascending order, because F is sorted.

F						L
\$	a_3	b_1	a_1	a_2	b_0	a_0
a_0	\$	a_3	b_1	a_1	a_2	b_0
a_1	a_2	b_0	a_0	\$	a_3	b_1
a_2	b_0	a_0	\$	a_3	b_1	a_1
a_3	b_1	a_1	a_2	b_0	a_0	\$
b_0	a_0	\$	a_3	b_1	a_1	a_2
b_1	a_1	a_2	b_0	a_0	\$	a_3

F is now completely described by two numbers per alphabet symbol: where its block starts and how long it is. Nothing else about F needs storing.

F costs one integer per alphabet symbol

F is a \$, then a block of as with ascending ranks, then a block of bs with ascending ranks.

	<i>F</i>	<i>L</i>
$C[\$] = 0$	\$	a_0
$C[a] = 1$	a_0	b_0
	a_1	b_1
	a_2	a_1
	a_3	\$
$C[b] = 5$	b_0	a_2
	b_1	a_3

$C[c] = \# \text{ characters in } T \text{ smaller than } c$

$C[\$] = 0$

$C[a] = 1$

$C[b] = 5$

On DNA that is four integers, whatever the length of the genome.

Which row begins with b_1 ?

F	L
\$	a_0
a_0	b_0
a_1	b_1
a_2	a_1
a_3	\$
b_0	a_2
b_1	a_3

- Skip the row beginning with \$ – 1 row
- Skip the rows beginning with a – 4 rows
- Skip b_0 – 1 row

Answer: row $1 + 4 + 1 = 6$, i.e. $C[b] + 1$.

No search. Two additions, because F is sorted and we know the block sizes.

The same question at genome scale

Say T has 300 **A**s, 400 **C**s, 250 **G**s and 700 **T**s, with $\$ < A < C < G < T$. Which row begins with **G**₁₀₀?

skip rows beginning with $\$$

1 row

skip rows beginning with A

300 rows

skip rows beginning with C

400 rows

skip the first 100 rows beginning with G

100 rows

Answer: row $1 + 300 + 400 + 100 = 801 = C[G] + 100$.

Undoing the transform

Start in row 0. Its F is \$, so the character just before the end of T is L[0].

<i>F</i>	<i>L</i>
\$	a ₀
a ₀	b ₀
a ₁	b ₁
a ₂	a ₁
a ₃	\$
b ₀	a ₂
b ₁	a ₃

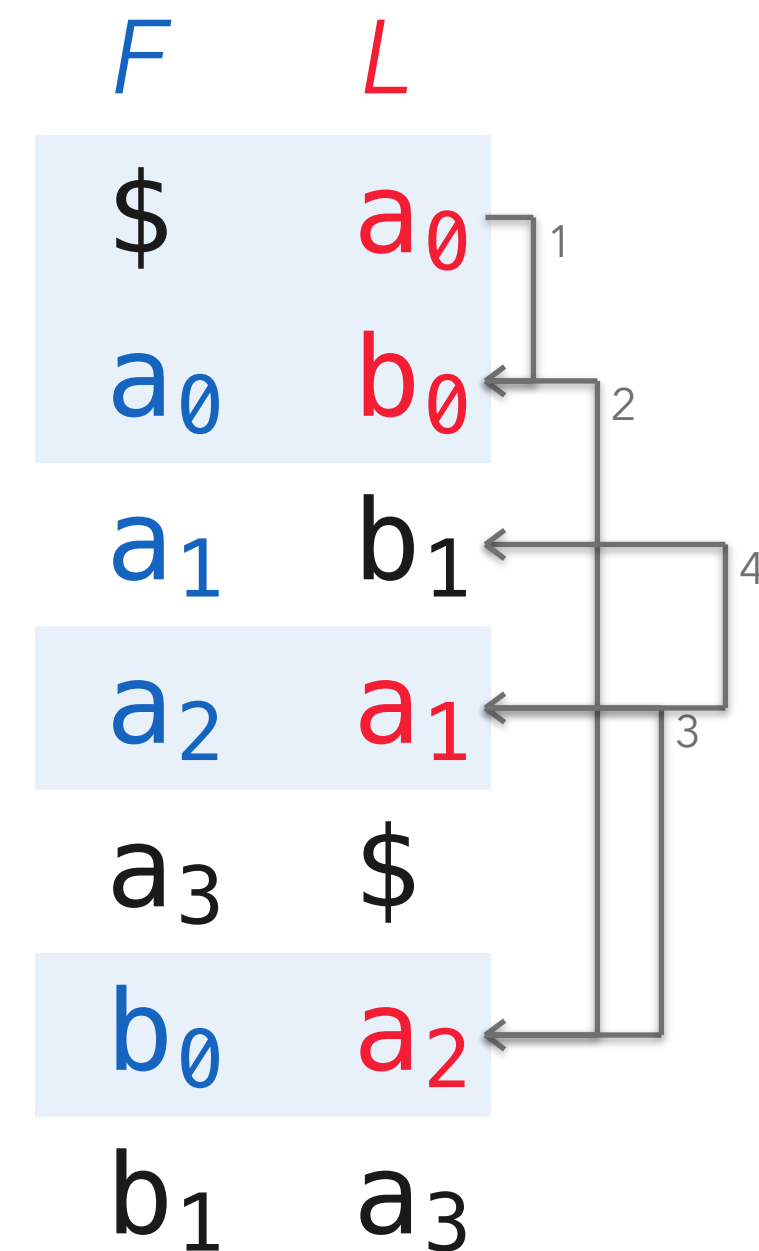
row **0** ends in **a** → jump to row **1**

row **1** ends in **b** → jump to row **5**

T so far (right to left): b a \$

Undoing the transform (continued)

Each LF step walks one character further to the left in T.



row **0** ends in **a** → jump to row **1**

row **1** ends in **b** → jump to row **5**

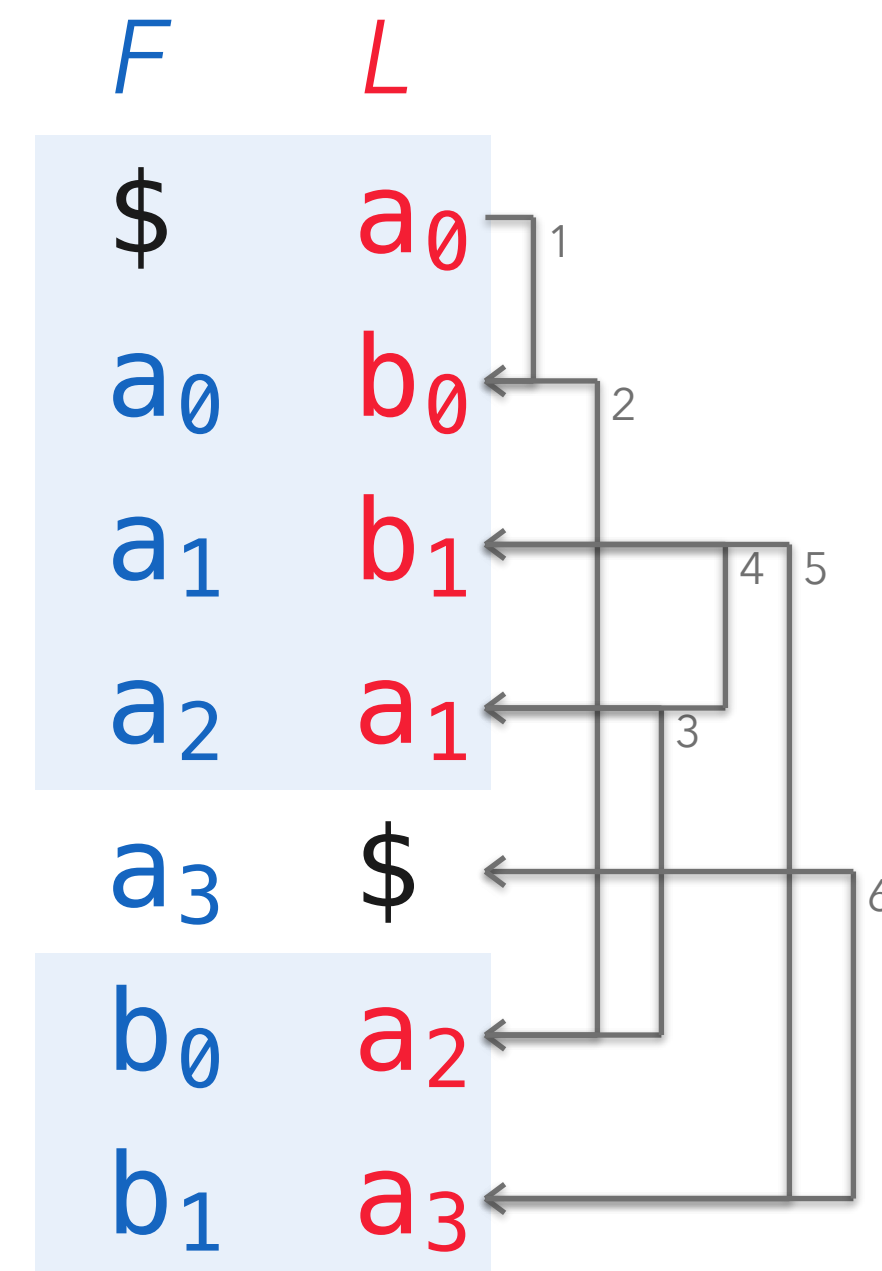
row **5** ends in **a** → jump to row **3**

row **3** ends in **a** → jump to row **2**

T so far (right to left): a a b a \$

Undoing the transform (continued)

Each LF step walks one character further to the left in T.



row **0** ends in **a** → jump to row **1**

row **1** ends in **b** → jump to row **5**

row **5** ends in **a** → jump to row **3**

row **3** ends in **a** → jump to row **2**

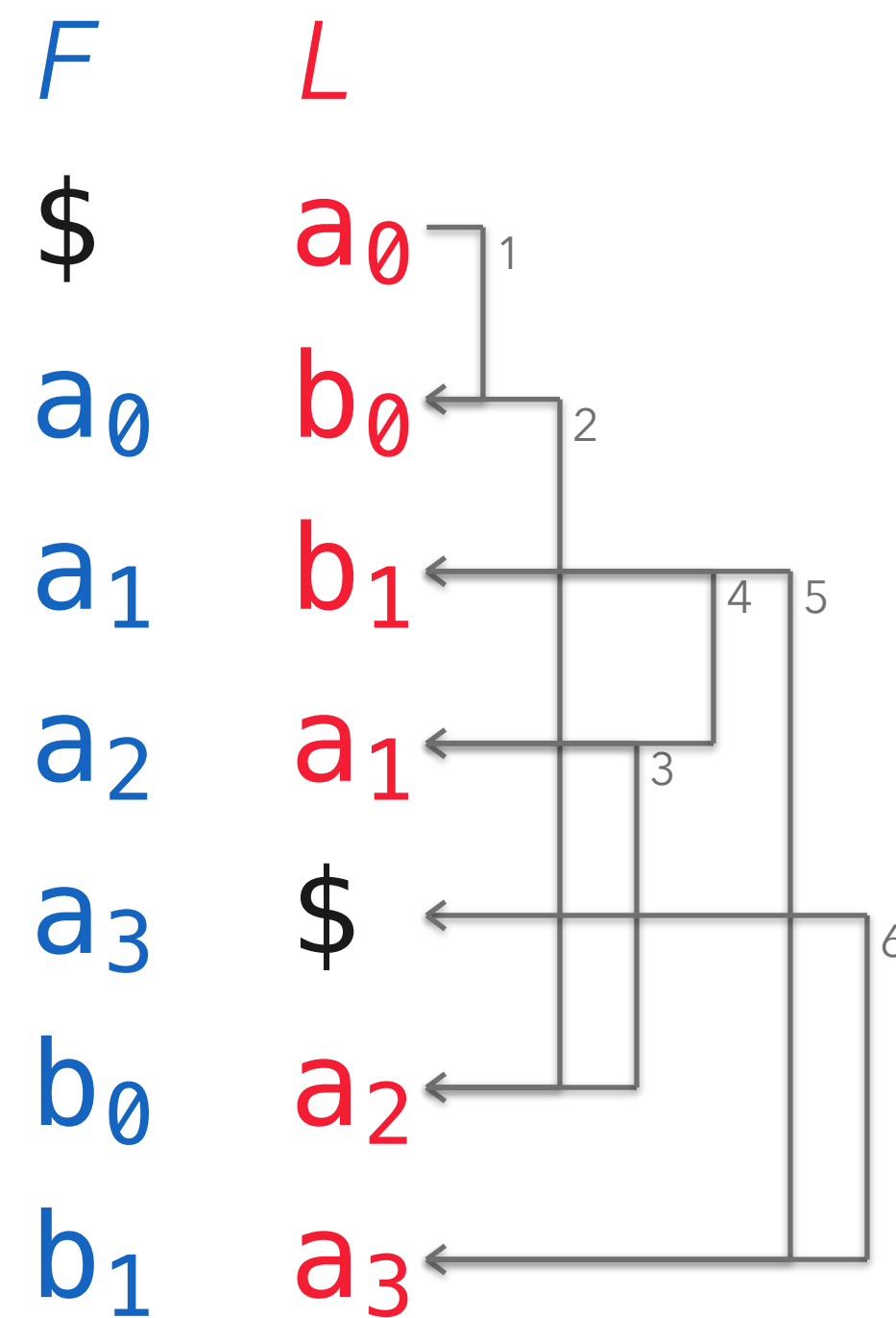
row **2** ends in **b** → jump to row **6**

row **6** ends in **a** → jump to row **4**

T so far (right to left): a b a a b a \$

The whole walk

Six LF steps recover T from right to left. No suffixes were stored.



T = a b a a b a \$

Reversing, in five lines

Occ(c, row) = # of cs in L[0 .. row-1]

reverse(L):

 C ← first-column offsets of L

 row ← 0; T ← "\$"

 while L[row] ≠ \$:

 c ← L[row]

 T ← c + T # prepend

 row ← C[c] + Occ(c, row) # the LF step

 return T

- Everything here is cheap except **Occ**. Counting cs by scanning L costs $O(m)$ per step, which makes reversal $O(m^2)$; storing every rank costs $m \cdot |\Sigma|$ integers, which defeats the point of the index.
- Fixing **Occ** is the subject of the fourth section. Hold the thought – it is the same operation the search will need.

Two down

Does it compress?

yes

L sorts characters by right-context, so it has runs.

Is it reversible?

yes

Repeated LF steps rebuild T from right to left.

Is it an index?

?

We can rebuild T. Can we search it without rebuilding it?

Backward search

Matching P right to left, one character at a time



Binary search is not available

Binary search on a suffix array works because we can read the suffix at any row and compare it with P . The FM index stores L , and $|\Sigma|$ integers from which F can be reconstructed – the columns in between are exactly what we threw away.

F							L
\$	a	b	a	a	b		a
a	\$	a	b	a	a		b
a	a	b	a	\$	a		b
a	b	a	\$	a	b		a
a	b	a	a	b	a		\$
b	a	\$	a	b	a		a
b	a	a	b	a	\$		a

not stored

We cannot ask what row 3 looks like. We have to search using the one column we kept, plus the $|\Sigma|$ integers that stand in for F – which means searching P backwards.

The idea

- Keep a **range of rows** whose rotations begin with the part of P matched so far. Because the matrix is sorted, those rows are always contiguous.
- Start with the shortest suffix of P – its last character – and extend one character to the left at a time.
- To extend by c , look at L inside the current range: those are precisely the characters that precede the current matches. Keep the c s, and apply LF to find where they land in F .
- Stop when P is exhausted (the range holds every match) or when the range becomes empty (P does not occur).

$$lo \leftarrow C[c] + occ(c, lo)$$

$$hi \leftarrow C[c] + occ(c, hi)$$

one step of backward search

Searching $P = aba$ · matching "a"

$P = ab\mathbf{a}$

	F	L
$[0, 7)$	\$	a_0
	a_0	b_0
	a_1	b_1
	a_2	a_1
	a_3	\$
	b_0	a_2
	b_1	a_3

$[1, 5)$

- Every row begins with something, so we start with $[0, 7)$.
- Rows beginning with a are $C[a] = 1$ through $C[a] + 4 = 5$.

Rows whose rotation begins with "a": $[1, 5)$ – 4 of them

Searching $P = aba$ · matching “ba”

$P = a\mathbf{ba}$

	F	L
	\$	a_0
$[1, 5)$	a_0	$\mathbf{b}_0$
	a_1	$\mathbf{b}_1$
	a_2	a_1
	a_3	\$
	$\mathbf{b}_0$	a_2
	$\mathbf{b}_1$	a_3

$[5, 7)$

- Inside $[1, 5)$, the characters of L are the ones immediately left of the current matches.
- The $\mathbf{b}$ s among them are occurrences $\mathbf{0}$ up to $\mathbf{2}$ of $\mathbf{b}$ in L .
- LF sends them to rows $C[\mathbf{b}] + 0 = \mathbf{5}$ through $\mathbf{7}$.

Rows whose rotation begins with “ba”: $[5, 7) - 2$
of them

Searching $P = aba$ · matching “aba”

$P = \mathbf{aba}$

F	L
\$	a_0
a_0	b_0
a_1	b_1
a_2	a_1
a_3	\$
[5, 7)	
b_0	a_2
b_1	a_3

- Inside **[5, 7)**, the characters of L are the ones immediately left of the current matches.
- The **a**s among them are occurrences **2** up to **4** of **a** in L .
- LF sends them to rows $C[a] + 2 = \mathbf{3}$ through **5**.

Rows whose rotation begins with “aba”: $[3, 5) - 2$
of them

The same range the suffix array would have given

Three LF steps, no comparisons against P beyond single characters, and no suffixes stored.

<i>F</i>	<i>L</i>	SA(T)	
\$	a_0	6	\$
a_0	b_0	5	a \$
a_1	b_1	2	a a b a \$
a_2	a_1	3	a b a \$
a_3	\$	0	a b a a b a \$
b_0	a_2	4	b a \$
b_1	a_3	1	b a a b a \$

[3, 5)

Two matches. The suffix array could also say where: offsets 0 and 3.
The FM index, as it stands, cannot – that is problem three.

Backward search, in six lines

|P| iterations, two Occ queries each

```
backward_search(P):
```

```
    lo ← 0; hi ← m
```

```
    for c in reversed(P):
```

```
        if c ∉ Σ: return (lo, lo)      # no such character in T
```

```
        lo ← C[c] + Occ(c, lo)
```

```
        hi ← C[c] + Occ(c, hi)
```

```
        if lo ≥ hi: return (lo, lo)    # empty: P does not occur
```

```
    return (lo, hi)                  # hi - lo occurrences
```

- With **Occ** in $O(1)$, counting the occurrences of P costs **$O(|P|)$** – independent of the length of the text.
- Compare lecture 6: binary search on a suffix array costs $O(|P| \log m)$, or $O(|P| + \log m)$ with the LCP interval tables. The FM index drops the $\log m$ entirely, and needs no comparison longer than one character.

When P does not occur

$P = bba$. The first two steps succeed; the third finds no b to extend with.

	F	L
	\$	a_0
	a_0	b_0
	a_1	b_1
	a_2	a_1
	a_3	\$
rows with prefix "ba" [5, 7)	b_0	a_2
	b_1	a_3

No b appears in L inside that range, so $\text{Occ}(b, lo) = \text{Occ}(b, hi)$ and the new range is empty. We stop and report zero occurrences – after three character steps, not after any search of T .

The same search on a longer string

$T' = a\ b\ a\ a\ b\ a\ a\ b\ a\ a\ b\ b\ b\ a\ a\ \$$

$P = a\ a\ b$

step	c	range before	Occ(c, lo)	Occ(c, hi)	range after
1	b	[0, 16)	0	6	[10, 16)
2	a	[10, 16)	5	9	[6, 10)
3	a	[6, 10)	2	5	[3, 6)

3 occurrences, at offsets 2, 5, 8 – which is exactly where “aab” sits in T' .

Three things are still wrong

1 Occ is a scan

Counting cs in L up to a row costs $O(m)$, and every step of every search pays it.

2 ranks are $m \cdot |\Sigma|$ integers

Storing Occ for every row and every symbol is ~50 GB on a genome: sixty times T itself, and four times the suffix array.

3 how many, not where

Backward search returns a range of rows. The suffix array is what turns rows into text offsets, and we dropped it.

All three are solved by adding something small and sparse – never by storing the matrix.

Making it fast

Three problems, three sparse tables



Problem 1: precompute Occ

Store, at every row i and every symbol c , the value $\text{Occ}(c, i)$ itself.

- $\text{Occ}(c, k)$ is then a single lookup: **$O(1)$** .
- But the table is **$m \times |\Sigma|$** integers. On a human genome that is 3.1 billion $\times 4 \times 4$ bytes ≈ 50 GB – 4 times the suffix array we were trying to beat.
- *So: keep the idea, store less of it.*

<i>F</i>	<i>L</i>	<i>Tally</i>	
		<i>a</i>	<i>b</i>
\$	a	0	0
a	a	1	0
a	b	2	0
a	b	2	1
a	b	2	2
a	b	2	3
a	\$	2	4
a	a	2	4
a	a	3	4
a	a	4	4
b	b	5	4
b	a	5	5
b	a	6	5
b	a	7	5
b	b	8	5
b	a	8	6

Checkpoints

Pick a period b and keep the tally only at every b -th row – here $b = 4$. Call those rows checkpoints.

- A lookup at a checkpoint is still one read.
- A lookup anywhere else: walk along L to the nearest checkpoint, counting c s as you pass them, and adjust.
- With checkpoints every b rows the walk is at most $b - 1$ steps – **$O(1)$** for fixed b – and the table shrinks by a factor of b .

F	L	Tally	
		a	b
\$	a	0	0
a	a		
a	b		
a	b		
a	b	2	2
a	b		
a	\$		
a	a		
a	a	3	4
a	a		
b	b		
b	a		
b	a	6	5
b	a		
b	b		
b	a		

A lookup that misses

What is $\text{Occ}(a, 10)$ – how many as appear in L before row 10?

- Row **10** is not a checkpoint. The nearest one above is row **8**.
- Its entry says $\text{Occ}(a, \mathbf{8}) = \mathbf{3}$.
- Rows **8** through **9** of L hold **2** more as. Scan them and add.

$$\text{Occ}(a, 10) = 3 + 2 = 5$$

<i>F</i>	<i>L</i>	<i>Tally</i>	
		<i>a</i>	<i>b</i>
\$	a	0	0
a	a		
a	b		
a	b		
a	b	2	2
a	b		
a	\$		
a	a		
a	a	3	4
a	a		
b	b		
b	a		
b	a	6	5
b	a		
b	b		
b	a		

The same thing, said with bit vectors

Keep one bit vector per alphabet symbol: a 1 wherever that symbol occurs in L , a 0 elsewhere. Then $\text{Occ}(c, k)$ is exactly $\text{rank}(V_c, k)$ – the number of 1s among the first k entries. The checkpoint table counts the same way, so there is one convention and no off-by-one.

bits for a	1	1	0	0	0	0	0	1	1	1	0	1	1	1	0	1
bits for b	0	0	1	1	1	1	0	0	0	0	1	0	0	0	1	0
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

a rank query

$$\text{rank}(a, 9) = 4$$

$$\text{rank}(b, 9) = 4$$

$$= \text{Occ}(a, 9), \text{Occ}(b, 9)$$

- Published rank structures answer in $\mathbf{O(1)}$ using $\mathbf{o(m)}$ bits *on top of* the vector. Our checkpoint table is the same idea but it is *not* $\mathbf{o(m)}$: at a fixed spacing it stores $\Theta(m)$ counters of $\log m$ bits each, which is why it shows up as a real line in the footprint. Getting the overhead down to $\mathbf{o(m)}$ needs a spacing that grows with m – the block/superblock scheme in the bonus section.
- For large alphabets one stores a wavelet tree instead of $|\Sigma|$ separate vectors, keeping the space at $m \log |\Sigma|$ bits rather than $m|\Sigma|$. For DNA the usual choice is simpler: pack L two bits per base and checkpoint popcounts over it.

Problem 3: a range of rows is not an answer

Backward search ends holding **[3, 5)**. A student wants “offsets 0 and 3”. The suffix array would say so immediately – but it is m integers, which is what we came here to avoid.

<i>F</i>	<i>L</i>	<i>SA</i>
\$	a_0	6
a_0	b_0	5
a_1	b_1	2
a_2	a_1	3
a_3	\$	0
b_0	a_2	4
b_1	a_3	1

Keeping the whole suffix array alongside the BWT costs 12 GB on a genome – it would dominate everything else in the index put together.

Sample the suffix array

Pick a period a . Keep an SA entry only when its value is a multiple of a – here $a = 4$.

- 4 of 16 entries survive – one in 4.
- Which rows survive is decided by the *text offset*, not by the row number, so the kept rows are scattered through the matrix.
- Sampling every a -th text position costs m/a integers. At $a = 32$ on a genome, 12 GB becomes **388 MB**.

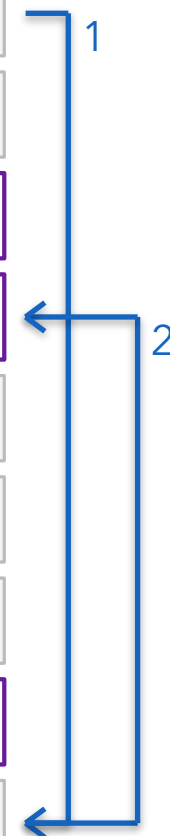
<i>F</i>	<i>L</i>	<i>SA</i>
\$	a_0	
a_0	a_1	
a_1	b_0	
a_2	b_1	
a_3	b_2	
a_4	b_3	8
a_5	\$	0
a_6	a_2	
a_7	a_3	
a_8	a_4	
b_0	b_4	12
b_1	a_5	
b_2	a_6	4
b_3	a_7	
b_4	b_5	
b_5	a_8	

Why multiples of a text offset, and not every 4th row? Because the next slide walks backwards through T , and a walk of a few steps has to be guaranteed to land on something kept.

Resolving an unsampled row

Row **3** was discarded. Apply LF: that walks one character to the left in T, so whatever offset we eventually land on, ours is that offset plus the number of steps we took.

<i>F</i>	<i>L</i>	<i>SA</i>
\$	a_0	
a_0	a_1	
a_1	b_0	
a_2	b_1	
a_3	b_2	
a_4	b_3	8
a_5	\$	0
a_6	a_2	
a_7	a_3	
a_8	a_4	
b_0	b_4	12
b_1	a_5	
b_2	a_6	4
b_3	a_7	
b_4	b_5	
b_5	a_8	



step 1: row **3** → row **11**

step 2: row **11** → row **6**

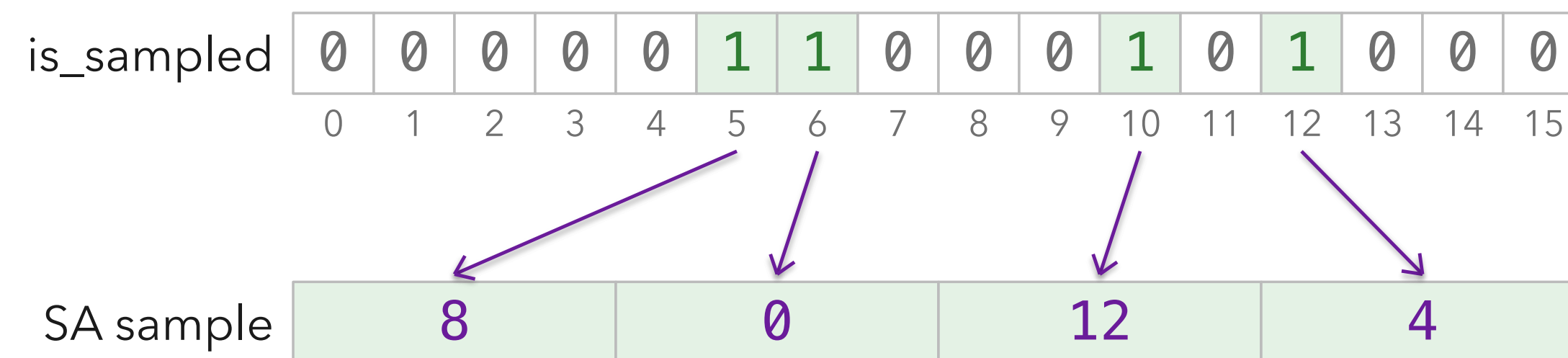
row **6** is sampled: SA = **0**

$$SA[3] = 0 + 2 = 2$$

At most $a - 1 = 3$ LF steps, because every a -th text position is kept – and that bound is independent of m .

Storing the sample without storing the gaps

A dictionary from row to offset would waste the very space we saved. Instead: one bit per row saying “kept”, and a dense array of the kept values in row order.



rank(is_sampled, i) = number of 1s among the first i entries

```
if is_sampled[i] = 0:  
    not stored – take an LF step  
else:  
    SA[i] = sample[ rank(is_sampled, i) ]
```

The same rank operation that made Occ fast now makes the lookup table dense. One primitive, two jobs.

Locating

$O(|P|)$ to count, then $O(a)$ per hit

```
locate(P):  
    lo, hi ← backward_search(P)  
    for row in lo .. hi-1:  
        steps ← 0  
        while is_sampled[row] = 0:  
            row ← C[L[row]] + Occ(L[row], row)  
            steps ← steps + 1  
        emit sample[rank(is_sampled, row)] + steps
```

- Counting is independent of how many matches there are. Reporting costs at most $a - 1$ LF steps per match – the sampling period a is the dial between index size and reporting speed.
- *Aligners set it by use case: $a = 32$ is typical for read mapping, where most patterns have few occurrences.*

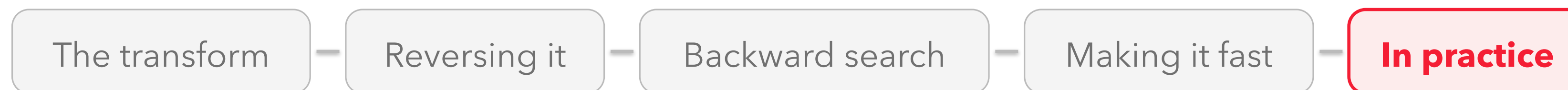
Three problems, solved

- 1 Occ is a scan**
checkpointed tally, or rank over bit vectors $O(1)$ time
- 2 ranks are $m \cdot |\Sigma|$ integers**
keep the tally only every b -th row $b = 128$ on a genome
- 3 rows, not offsets**
sample the suffix array, walk LF to a sampled row $a = 32$ on a genome

Each fix adds something proportional to m with a small constant – and nothing proportional to $m \log m$.

In practice

What it costs, and why every aligner uses one



What the index actually contains

component	what it is	size
F	block start per alphabet symbol (the C array)	$ \Sigma $ integers
L	the transform itself	m characters
checkpoints	tally of each symbol every b-th row	$m \cdot \Sigma / b$ integers
SA sample	suffix-array value every a-th text position	m / a integers
is_sampled	which rows kept a value (plus its rank index)	m bits + $o(m)$

- The matrix is never stored. Neither are the suffixes, nor the rotations, nor the full suffix array.
- a and b are the two knobs. Turning them *up* shrinks the index and slows queries, along a curve that is remarkably flat in the useful range.

A human genome

$m \approx 3.1$ billion bases, $|\Sigma| = 4$, $a = 32$, $b = 128$, 4-byte integers.



Total ≈ 1.9 GB – against ~ 12 GB for the plain suffix array (lecture 6) and ~ 49 GB for the suffix tree (lecture 5). And it fits in the memory of a laptop.

Two lines are easy to forget: the checkpoint tally is kept per symbol, so the $|\Sigma|$ factor is real; and sampling by text position needs its own bit vector, whose rank index is folded into the 388 MB above.

The four structures, finally

structure	space (human genome)	count P	locate one hit
suffix trie	$\Theta(m^2)$ – impossible	$O(P)$	$O(1)$
suffix tree	~49 GB	$O(P)$	$O(1)$
suffix array	~12 GB	$O(P + \log m)^*$	$O(1)$
FM index	~1.9 GB	$O(P)$	$O(a)$

** with the Llcp / Rlcp tables, bringing the total to ~37 GB; plain binary search is $O(|P| \log m)$.*

The FM index is the only row where the space column is smaller than four bytes per base – and it gets there by storing a permutation of the text instead of a set of pointers into it.

Building the index

- The naive route – write every rotation, sort them – is $O(m^2 \log m)$ time and $O(m^2)$ space. For a genome it is not merely slow; the matrix does not fit on any disk.
- Via the suffix array: build SA in $O(m)$ (lecture 6's DC3), then read off L in one pass. Correct and linear, but the constants and the peak memory are large.
- Direct $O(m)$ BWT algorithms avoid materialising SA. Blockwise suffix sorting sorts the suffixes in chunks, trading a constant factor of time for a working set the user chooses – which is what makes “index a genome overnight on a workstation” a true statement.
- *Kärkkäinen J. Fast BWT in small space by blockwise suffix sorting. Theoretical Computer Science 387(3):249-257, 2007.*

What to remember

- BWT(T) is the last column of the sorted rotation matrix, and equally “the character left of each suffix, in suffix-array order”. The second definition is the one you compute with.
- LF mapping: the i th c in L is the i th c in F . It holds because both columns are sorted by the same right-contexts.
- One LF step walks one character left in T . Iterating it reverses the transform; iterating it over a shrinking row range searches the transform. Same step, two uses.
- **Backward search counts occurrences in $O(|P|)$, with no dependence on m at all.**
- Everything expensive is made cheap by sampling: tallies at every b th row, suffix-array values at every a th position, and rank to make the sparse tables dense again.

Bonus material

Not on the exams. Worth knowing about.

The FM index as it was actually published

What we built stores L uncompressed – m characters, or $2m$ bits on DNA. Ferragina and Manzini's index stores a **compressed** L and still answers Occ in constant time, which is the part that looks impossible.

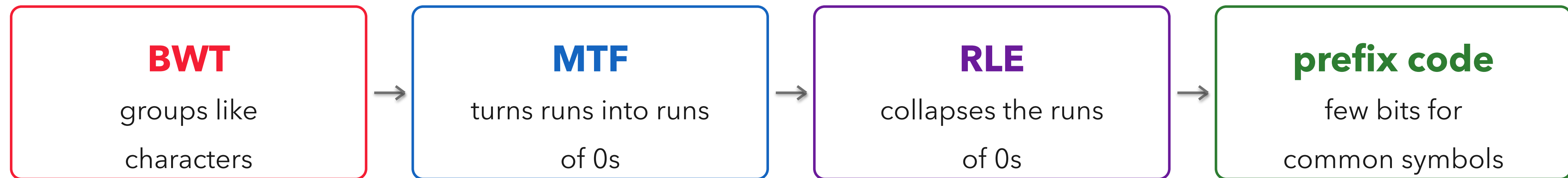
$$m \cdot O(H_k(T)) + o(m) \text{ bits, simultaneously for all } k = o(\log_\sigma m)$$

$H_k(T)$ = the k th-order empirical entropy of T , in bits per character

- What shrinks is the *constant per character*, not the asymptotic order: the index is still $\Theta(m)$, but for a compressible string it occupies **fewer bits than T's own plain encoding** – while still answering queries on it.
- This is what “opportunistic data structure” means in the title: it takes advantage of structure in the data when there is any, and degrades gracefully when there is not.
- *Ferragina P, Manzini G. Opportunistic data structures with applications. FOCS 2000, 390–398.*

Compressing L

BWT groups like characters together, which is exactly what the next three stages are good at.



`PrefixCode( RLE( MTF( BWT(T) ) ) )` – each stage makes the next one's job easier, and none of them loses anything.

Which is, give or take, bzip2.

Move-to-front coding

Emit each symbol's index in the current list, then move that symbol to the front.

alphabet \$ d g o w

input d o \$ o o d w g

output 1 3 2 1 0 2 4 4

<i>list before</i>	<i>symbol</i>	<i>code</i>
\$dgow	d	1
d\$gow	o	3
od\$gw	\$	2
\$odgw	o	1
o\$dgw	o	0
o\$dgw	d	2
do\$gw	w	4
wdo\$g	g	4

- A run of the same symbol becomes a 0 after the first one – so BWT's runs become runs of 0s, which run-length coding eats.
- Common symbols drift to the front and keep small codes; rare symbols sink. The output is skewed towards 0, which is what a prefix code wants.

Move-to-front decoding

Read a code, emit the symbol at that index, move it to the front. The list is rebuilt as we go – nothing else is sent.

alphabet \$ d g o w

input 1 3 2 1 0 2 4 4

output d o \$ o o d w g

<i>list before</i>	<i>code</i>	<i>symbol</i>
\$dgow	1	d
d\$gow	3	o
od\$gw	2	\$
\$odgw	1	o
o\$dgw	0	o
o\$dgw	2	d
do\$gw	4	w
wdo\$g	4	g

Decoding needs only the codes and the starting alphabet. The list state is reconstructed, never transmitted.

But now Occ is hard again

Occ(c, p) asks how many cs are in the first p characters of L . If L is stored compressed, we cannot index into it
– the pth character is not at any predictable place.

superblock 0				superblock 1				superblock 2			
B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11

- Cut L into **buckets** of $\ell = \Theta(\log u)$ characters, each compressed *independently* so that it decodes on its own, and group every ℓ buckets into a superblock. (u is the compressed length; ℓ is a block length, not the column L .)
- Each **superblock** stores, per symbol, the count from the start of the string. Each **bucket** stores, per symbol, the count since its superblock began.
- A third table answers “how many cs in the first k characters of a bucket with this content” by lookup – a bucket is short enough that every possible content can be tabulated once and shared by every bucket.

$$\text{Occ}(c, p) = \text{superblock count} + \text{bucket count} + \text{table lookup inside the bucket} \\ \text{– three reads, } O(1).$$

Why the bookkeeping is affordable

table	entries	bits each	total
superblock counts	u / ℓ^2	$ \Sigma \cdot \log u$	$O(u / \log u)$
bucket counts	u / ℓ	$ \Sigma \cdot \log \ell$	$O(u \log \log u / \log u)$
in-bucket lookup	$2^\ell \cdot \ell \cdot \Sigma $	$\log \ell$	$O(u^c), c < 1$

- With $\ell = \Theta(\log u)$ every row is **$o(u)$** – vanishing beside the compressed string itself.
- The in-bucket table is shared by every bucket, which is the trick: there are only 2^ℓ distinct bucket contents, and with $\ell = \Theta(\log u)$ that is a sub-linear function of u .
- *This is the same three-level pattern as a rank/select structure, and the same pattern as the checkpoints earlier – a coarse index, a fine index, and a table for the last few characters. The difference is that here the spacing grows with u , which is what finally makes the overhead $o(u)$ rather than $\Theta(u)$ – see the caveat on the bit-vector slide.*

Where this goes

- The FM index is the backbone of Bowtie, BWA and almost every short-read aligner written since 2009.
- Its descendants index collections rather than single strings: the r-index, whose space depends on the number of *runs* in the BWT rather than its length, makes indexing thousands of genomes practical, because a pangenome's BWT has far fewer runs than characters.
- Which is the same observation we started with – BWT sorts characters by right-context, so similar strings produce long runs – pushed as far as it will go.

Next: read alignment on top of an FM index.